Isaac ROS Environment Setup (Jetson Orin Nano)

Isaac ROS Environment Setup (Jetson Orin Nano)

1. Scope of Application
2. Obtaining and Checking Files
    Disk Space Requirements
3. Importing the Docker Image
4. Extracting the Course Workspace
5. Starting the Container
    5.1 Preparing Fast DDS and the Terminal Environment
    5.2 Creating the Startup Script
6. Checking the ROS Environment
7. Model Preparation Rules
8. Optional: Deleting the Archives After Deployment Is Confirmed
9. Usage Notes
10. Next Steps

1. Scope of Application

This tutorial is intended for deploying the Isaac ROS environment and the case workspace provided by the course on a Jetson Orin Nano 8GB. Before deployment, confirm that the device matches the table below; for other Jetson models or JetPack versions, image compatibility must be confirmed first.

> Note: The remaining space of the factory image system is insufficient to set up the Isaac ROS environment. Use the dedicated Isaac image provided in the materials (the environment is already configured), or use another third-party system image with sufficient space to set up the environment. For a third-party system image, refer to the following tutorials: 【Chapter 2 Mainboard Basics: 02. Writing the Jetson Clean System】 -> 【Chapter 2 Mainboard Basics: 03. Installing the Jetson Environment】 -> 【Chapter 8 Docker Basics: 02. Docker Installation】

Item	Requirement
Mainboard	Jetson Orin Nano 8GB, aarch64
JetPack / L4T	JetPack 7.2.1 / L4T R39.2.1
System	Ubuntu 24.04
Docker environment	Installed and runnable by the current user
NVIDIA Container Toolkit	Installed
Disk space	Must hold the imported image, the extracted workspace, and runtime headroom at the same time; verify with the formula in this section

The environment consists of two parts: the Docker image provides ROS 2 Jazzy and the Isaac ROS software; the workspace archive provides the course models, data, and case files. The cases can only be run after both parts are deployed.

2. Obtaining and Checking Files

Disk Space Requirements

Before importing, the partition holding the Docker data directory and the partition holding the workspace target should satisfy the following respectively:

- If both are on the same partition: the remaining space must be at least **65.3GB + the extracted workspace size + 20GB runtime headroom**.
- If the two are on different partitions: reserve at least **85GB** on the partition holding the Docker data directory; reserve at least **the extracted workspace size + 10GB** on the partition holding the workspace

After confirming that the space is sufficient, place the following two files in the `/home/jetson/` directory of the Jetson:

File	Purpose
`my-isaac-ros-slam-4.6-all.tar.gz`	Isaac ROS Docker image, the archive is about 24.3GB; the Docker image after import is about 65.3GB
`isaac_ros_workspaces.tar.gz`	Case workspace, models, and data; the archive size and the extracted size vary with the delivered content, subject to the actual files and the delivery list

If the files are on another computer, upload them from the terminal of that computer; replace `<Jetson_IP>` with the actual IP of the Jetson:

```
scp my-isaac-ros-slam-4.6-all.tar.gz jetson@<Jetson_IP>:/home/jetson/
scp isaac_ros_workspaces.tar.gz jetson@<Jetson_IP>:/home/jetson/
```

Then confirm the files, the system, and the disk space on the Jetson:

```
uname -m
cat /etc/nv_tegra_release
docker --version
docker info --format 'DockerRootDir={{.DockerRootDir}} DefaultRuntime=
{{.DefaultRuntime}} Runtimes={{json .Runtimes}}'
df -h /var/lib/docker /home/jetson
ls -lh /home/jetson/my-isaac-ros-slam-4.6-all.tar.gz
/home/jetson/isaac_ros_workspaces.tar.gz
```

The device should be `aarch64`, and the L4T version should be `R39.2.1`. The Docker runtime list should contain `nvidia`.

3. Importing the Docker Image

Execute on the Jetson host:

```
docker load -i /home/jetson/my-isaac-ros-slam-4.6-all.tar.gz
docker image inspect my-isaac-ros-slam:4.6-all --format 'OS={{.Os}} Arch=
{{.Architecture}}'
docker images | grep my-isaac
```

After the import is complete, the image `my-isaac-ros-slam:4.6-all` should be visible, with the architecture `arm64`. Loading the image takes some time; do not close the terminal or delete the archive in the middle.

Expected output:

```
OS=linux Arch=arm64
my-isaac-ros-slam:4.6-all    xxxxxxx   about 65.3GB
```

4. Extracting the Course Workspace

First view the directory structure in the archive:

```
tar -tzf /home/jetson/isaac_ros_workspaces.tar.gz | head -n 30
```

The archive should contain the complete `workspaces/isaac_ros-dev/` workspace, including at least:

- `isaac_ros_assets/`: course models and rosbag data
- `launch/`: case launch files
- `rviz/`: RViz configuration
- `scripts/` and the case helper scripts in the root of the workspace
- `fastdds_no_shm.xml`: the Fast DDS configuration used by the course

If the list is missing the directories or files above, obtain the complete workspace archive from the course provider first. Select the corresponding command according to the result of `tar -tzf`. If the paths in the archive start with `workspaces/isaac_ros-dev/`, extract to `/home/jetson`:

```
mkdir -p /home/jetson/workspaces
tar -xzf /home/jetson/isaac_ros_workspaces.tar.gz -C /home/jetson
test -d /home/jetson/workspaces/isaac_ros-dev/isaac_ros_assets
test -d /home/jetson/workspaces/isaac_ros-dev/launch
test -d /home/jetson/workspaces/isaac_ros-dev/rviz
```

If the paths in the archive start directly with `isaac_ros-dev/`, extract to `/home/jetson/workspaces` and check:

```
mkdir -p /home/jetson/workspaces
tar -xzf /home/jetson/isaac_ros_workspaces.tar.gz -C /home/jetson/workspaces
test -d /home/jetson/workspaces/isaac_ros-dev/isaac_ros_assets
test -d /home/jetson/workspaces/isaac_ros-dev/launch
test -d /home/jetson/workspaces/isaac_ros-dev/rviz
```

Before extracting, confirm that the target directory does not exist or contains no files that need to be kept; if a check fails, verify the directory structure of the archive first and do not continue starting the container.

5. Starting the Container

5.1 Preparing Fast DDS and the Terminal Environment

The workspace is located in a host-mounted directory, so a configuration file placed in the workspace is retained across container restarts. First check whether the workspace already has `fastdds_no_shm.xml`; if not, create it on the Jetson host:

```
cat > /home/jetson/workspaces/isaac_ros-dev/fastdds_no_shm.xml <<'EOF'
<?xml version="1.0" encoding="UTF-8" ?>
<profiles xmlns="http://www.eprosima.com/XMLSchemas/fastRTPS_Profiles">
  <transport_descriptors>
    <transport_descriptor>
      <transport_id>udp_only</transport_id>
      <type>UDPv4</type>
    </transport_descriptor>
  </transport_descriptors>
  <participant profile_name="udp_only_participant" is_default_profile="true">
    <rtps>
      <userTransports>
        <transport_id>udp_only</transport_id>
      </userTransports>
      <useBuiltinTransports>false</useBuiltinTransports>
    </rtps>
  </participant>
</profiles>
EOF
```

The Fast DDS configuration uses the UDP transport and is the configuration of the container environment of this course. The startup script generates `/tmp/.bashrc` each time the container starts, which only contains the ROS and workspace environment settings; there is no need to create another environment file on the host, and the user configuration in the image is not modified.

5.2 Creating the Startup Script

On first use, create the startup script on the Jetson host:

```
cat > /home/jetson/run_isaac.sh <<'EOF'
#!/bin/bash
set -e

ISAAC_ROS_WS="$HOME/workspaces/isaac_ros-dev"

if [[ ! -d "$ISAAC_ROS_WS" ]]; then
  echo "The workspace does not exist: $ISAAC_ROS_WS" >&2
  exit 1
fi
if [[ ! -f "$ISAAC_ROS_WS/fastdds_no_shm.xml" ]]; then
```

```
  echo "The Fast DDS configuration file does not exist:
$ISAAC_ROS_WS/fastdds_no_shm.xml" >&2
  exit 1
fi

docker run --rm -it \
  --runtime=nvidia --gpus all \
  --network host --ipc host --privileged \
  -u 1000:1000 -e HOME=/tmp \
  -e DISPLAY="$DISPLAY" -e QT_X11_NO_MITSHM=1 \
  -v /tmp/.X11-unix:/tmp/.X11-unix:rw \
  -v "$ISAAC_ROS_WS:/workspaces/isaac_ros-dev" \
  --entrypoint /bin/bash \
  my-isaac-ros-slam:4.6-all \
  -c '
cat > /tmp/.bashrc <<'"'"'BASHRC_EOF'"'"'
source /opt/ros/jazzy/setup.bash
export ISAAC_ROS_WS=/workspaces/isaac_ros-dev
export FASTRTPS_DEFAULT_PROFILES_FILE=/workspaces/isaac_ros-
dev/fastdds_no_shm.xml
BASHRC_EOF
exec /bin/bash --rcfile /tmp/.bashrc -i
'
EOF
```

```
chmod +x /home/jetson/run_isaac.sh
```

The script is written for the `jetson` user with UID/GID `1000:1000`; other users should run `id` first and then modify the `-u` parameter according to the actual result. When the container starts, the ROS and workspace environment is written to `/tmp/.bashrc`, and an interactive Bash is started with that file through `--rcfile`. `--privileged` is part of the current device configuration of the course and is not recommended to be applied directly to containers for other purposes.

Graphical cases must be started from the Jetson desktop terminal. First allow the local Docker container to access the current desktop, then run the startup script:

```
xhost +local:docker
cd /home/jetson
./run_isaac.sh
```

After all graphical programs are closed, the authorization can be revoked:

```
xhost -local:docker
```

Do not use `xhost +` to disable all graphical access control, and do not modify the global Docker runtime by yourself. If an error is reported at startup, record the complete error message and check the image, the workspace, and the desktop session configuration.

After the container starts, open another Jetson terminal to view the container name:

```
docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
```

When an additional container terminal is needed, execute on the Jetson host:

```
docker exec -it <container name> bash
```

Replace `<container name>` with the name shown by `docker ps`.

6. Checking the ROS Environment

When entering the container with the startup script of this tutorial, the ROS and Fast DDS environment is loaded automatically by `/tmp/.bashrc`. Execute the following commands inside the container to check:

```
echo "$ROS_DISTRO"
echo "$ISAAC_ROS_WS"
echo "$FASTRTPS_DEFAULT_PROFILES_FILE"
test -f "$FASTRTPS_DEFAULT_PROFILES_FILE"
test -d "$ISAAC_ROS_WS/isaac_ros_assets"
test -d "$ISAAC_ROS_WS/launch"
```

The expected ROS distribution is `jazzy`, the workspace is `/workspaces/isaac_ros-dev`, and the Fast DDS configuration file exists. Then check the core package:

```
ros2 pkg prefix isaac_ros_visual_slam
```

The expected package path is located in `/opt/ros/jazzy`. If the container is started in another way, `source /opt/ros/jazzy/setup.bash` must be executed manually in each new terminal, and `ISAAC_ROS_WS` and `FASTRTPS_DEFAULT_PROFILES_FILE` must be set.

7. Model Preparation Rules

Case 5 uses the TensorRT plan in the `optimized_deployable_shuffleseg_unet_amr_v1.0` directory, which is loaded directly by the launch file; case 7 uses the Triton model `peoplenet`. Cases 1–4, 6, 8, and 9 do not need to be switched in this way. Before switching, stop the current case first, check the two directories inside the container, and then execute as needed:

```
cd /workspaces/isaac_ros-dev/isaac_ros_assets
ls models/triton/ models_pending/

# Move all current models under models/triton/ back to the pending directory
mv models/triton/* models_pending/

# Replace <model name> with the corresponding directory name above
mv models_pending/<model name> models/triton/
ls models/triton/
```

`mv models/triton/*` moves all models in the active directory away, so execute it only after confirming that no other model needs to be retained; if the target model is already in the active directory, no switch is required. Before moving, confirm that there is no directory with the same name in pending; if the command reports an error, check the directories first and do not hide the error with `2>/dev/null`. The CenterPose FP32 plan of case 8 is specified by the startup script and is not switched here.

8. Optional: Deleting the Archives After Deployment Is Confirmed

After confirming that the image has been imported successfully, the workspace has been extracted, and the container and cases can start normally, the two archives can be deleted if the installation packages do not need to be retained. This operation does not delete the imported Docker image or the extracted workspace files; keep the archives while the deployment has not been confirmed successful.

```
rm /home/jetson/my-isaac-ros-slam-4.6-all.tar.gz
rm /home/jetson/isaac_ros_workspaces.tar.gz
```

9. Usage Notes

- Run only one high-load case at a time; start the next one after the current case is finished.
- Start and stop the nodes of each case according to its operation instructions, and exit normally with `Ctrl+C` in the corresponding terminal; do not terminate processes in batches with `clean_ros` or a broad `pkill`.
- After the container exits, the container itself is deleted automatically; the Docker image and the files in `/home/jetson/workspaces/isaac_ros-dev/` on the host are retained.
- If `groups: cannot find name for group ID` is displayed but the container, the ROS nodes, and the case checks are normal, this prompt usually does not affect operation.
- If RViz or rqt cannot display, confirm that it was started from the Jetson desktop terminal and check whether the desktop graphical authorization has been set.
- If Docker, image import, or workspace checks fail, record the complete error message and contact the course provider; do not delete the delivery archives or modify the system-level Docker configuration.

10. Next Steps

After the environment check passes, run the course content by referring to the operation instructions of cases 1–9 in the directory of this chapter.